Technology Stack

Every language, library, algorithm and data structure in the project, and why each one is there.

At a glance

Layer	Choice	Version
Language	Python	3.9+ (developed on CPython 3.14)
GUI toolkit	Tkinter / ttk	Standard library
Search	A* over box pushes	Hand-written
Heuristic	Hungarian assignment on pull distances	Hand-written
Test framework	unittest	Standard library
Runtime dependencies	None	--
Version control	Git	--

Why no third-party packages

The entire program runs on a stock Python install. That is a deliberate constraint, not an oversight:

- It runs anywhere Python runs -- a lab machine, an examiner's laptop, a locked-down desktop -- with no pip, no network and no virtual environment.
- Nothing can break through a dependency upgrade.
- Every algorithm in the project is visible and reviewable rather than hidden behind a library call, which is the point of the exercise.

The documents you are reading were produced with reportlab and python-pptx, but those are **build-time only** and are not imported anywhere in the program.

Standard library modules used

Module	Used for
tkinter, tkinter.ttk	The window, canvas drawing, themed widgets, keyboard binding and the replay timer
heapq	The A* frontier as a binary min-heap
collections.deque	Breadth-first queues for flood fill, pull distances and walk paths
dataclasses	Frozen, hashable State and Push value objects; the mutable Result
threading	Running the search off the UI thread; Event for cancellation

queue	Handing the finished result back to the Tk main loop safely
time	perf_counter for the time budget and reported timings
pathlib	Locating the bundled level pack
typing	Type hints throughout
unittest	The test suite
html, textwrap	Documentation build only

Algorithms

Algorithm	Where	Role
A* search	solver.solve	Finds a push-optimal plan
Hungarian algorithm	solver.min_cost_assignment	Exact minimum-cost box-to-target matching for the heuristic
Reverse (pull) BFS	Analysis._pull_distances_from	Push distance from every square to each target
Flood fill / BFS	Board.reachable, Board._flood_fill	Worker reachability, state normalisation, enclosure check
BFS shortest path	Board.walk_path	Turns consecutive pushes into keystrokes
Freeze-deadlock recursion	Analysis._is_frozen	Detects mutually immovable boxes

Data structures

Structure	Why it was chosen
frozenset of coordinates	Box positions: hashable, so a state drops straight into a dict or set; set algebra makes a push one expression
Binary heap (heapq)	Frontier ordered by g + h; O(log n) push and pop
Hash set	Closed set of normalised states; O(1) duplicate rejection
Dict of dicts	Pull distances, one table per target
Frozen dataclass	State and Push: immutable, hashable, cheap to compare
Insertion counter	Heap tie-break, so states are never compared to each other

Complexity

Operation	Cost	Note
Pull-distance tables	O(targets x floor)	Once per solve, microseconds

Heuristic per node	O(b^3), b = boxes	Hungarian; negligible beside expansion
Successors per node	O(b x floor)	Reachability flood fill dominates
Deadlock test	O(b) amortised	Only the box that moved is checked
Search	Exponential in boxes	PSPACE-complete problem; pruning is what makes it finish

Measured performance

Level	Boxes	Pushes	Moves	Positions examined	Deadlocks pruned	Time
First Push	1	1	2	2	3	0.00s
Side by Side	2	2	7	3	5	0.00s
Around the Pillar	2	2	11	3	5	0.00s
The Hallway	2	6	27	7	6	0.00s
Long Way Round	1	9	22	10	7	0.00s
Crossroads	4	4	18	5	4	0.00s
The Vault	4	4	33	5	0	0.00s
Pinwheel	4	4	12	5	4	0.00s
Storage Room	4	20	78	36	57	0.01s
The Depot	3	21	79	39	43	0.00s
Tight Fit	4	10	49	11	38	0.00s
Warehouse Shift	5	28	117	5,539	11,472	2.01s

Measured when this document was generated, single-threaded CPython.

Concurrency model

One worker thread runs the search; the Tk main loop owns every widget. They communicate two ways, both one-directional:

- **Cancellation** -- a `threading.Event` the search polls every 2,000 nodes, so Stop and level switches take effect promptly.
- **Results** -- a `queue.Queue` the Tk main loop drains on a 60 ms timer.
- **Progress** -- a plain integer the worker overwrites and the timer reads, because calling into Tk from another thread is not safe.

Testing

Aspect	Approach
Framework	unittest, standard library, no plugins
Count	25 tests, about 4 seconds
Oracle	Every solution is replayed against the rules from the start position
Property checked	Heuristic admissibility: the start estimate never exceeds the optimal push count found
Fixtures	Hand-built trap positions for deadlock detection; malformed levels for the parser

Platform support

Windows, macOS and Linux, wherever CPython 3.9+ and Tkinter are present. No platform-specific code, no file-system assumptions beyond locating the bundled level pack relative to the package.